

Juan Pablo Cuervo Contreras

Carné: 1000100656

David Felipe Ortiz Salcedo

Carné: 1000100448

ESCUELA COLOMBIANA DE INGENIERÍA DESARROLLO ORIENTADO POR OBJETOS

Interfaz gráfica 2026-1 — Laboratorio 5/6

OBJETIVO

Evidenciar el logro de los siguientes resultados de aprendizaje:

1. Desarrollar una mini aplicación gráfica considerando el patrón MVC.
2. Implementar el esquema de manejo de eventos con clases anónimas
3. Experimentar el comportamiento de las ventanas [JFrame](#), [JDialog](#) y [JOptionPane](#)
4. Seleccionar los lienzos más apropiados para un diseño: [JPanel](#) y [JTabbedPane](#)
5. Revisar las posibilidades de los estilos: [FlowLayout](#), [BorderLayout](#) y [GridLayout](#)
6. Apropiar algunos componentes básicos: [JLabel](#), [JTextField](#), [JButton](#), [JMenuBar](#)
7. Apropiar algunos componentes especiales: [JFileChooser](#) y [JColorChooser](#)
8. Experimentar las prácticas

Testing [Acceptance tests](#) are run often and the score is published.

Testing. When [a bug is found](#) tests are create.

ENTREGA

- Incluyan en un archivo [.zip](#) los archivos correspondientes al laboratorio. El nombre debe ser los apellidos de los dos miembros del equipo ordenados alfabéticamente y separados por un guion. (Casallas-Duarte) Publiquen el avance al
- final de la sesión y la versión definitiva en la fecha indicada, en los espacios correspondientes.

CONTEXTO

Reto: Implementar el juego **EasySokoban**

El propuesto por ustedes SokobanGUI	El acordado en laboratorio Sokoban
Vista-Controlador	Modelo

El trabajo debe hacerse desde CONSOLA

Para la capa de presentación NO deben hacer diagramas de diseño ni pruebas de unidad

PARTE I. Maqueta

A Directorios [En [lab05.doc](#)]

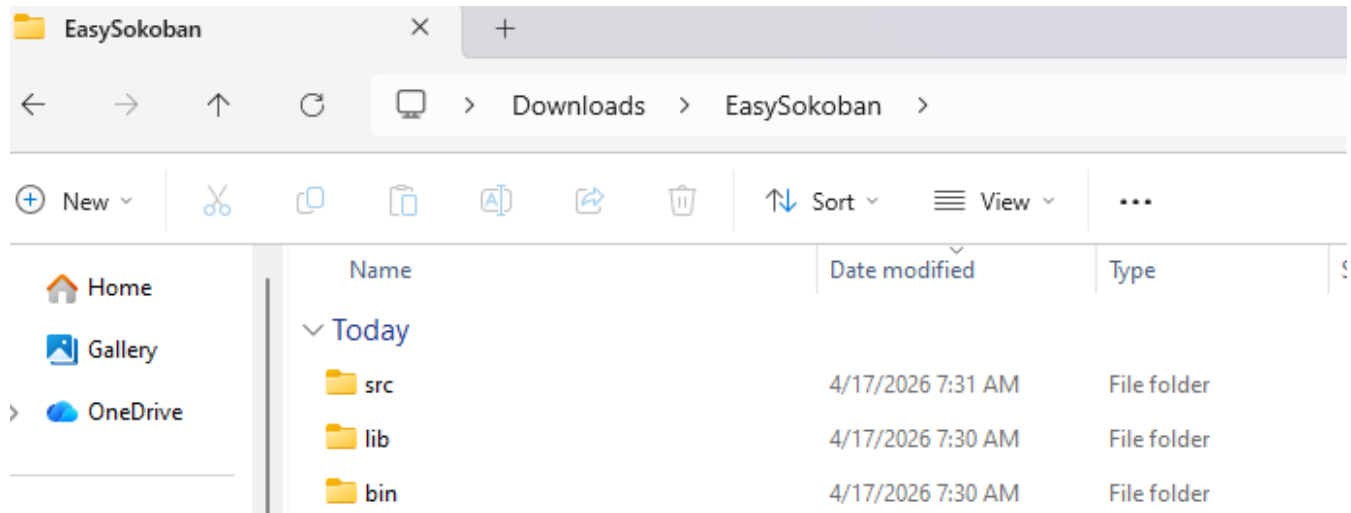
El objetivo de este punto es preparar los directorios para el juego **EasySokoban**

1. Preparen un directorio llamado **EasySokoban** con los directorios [src](#), [bin](#) y [lib](#); y los subdirectorios para presentación, dominio y pruebas de unidad. Capturen una pantalla con la estructura.

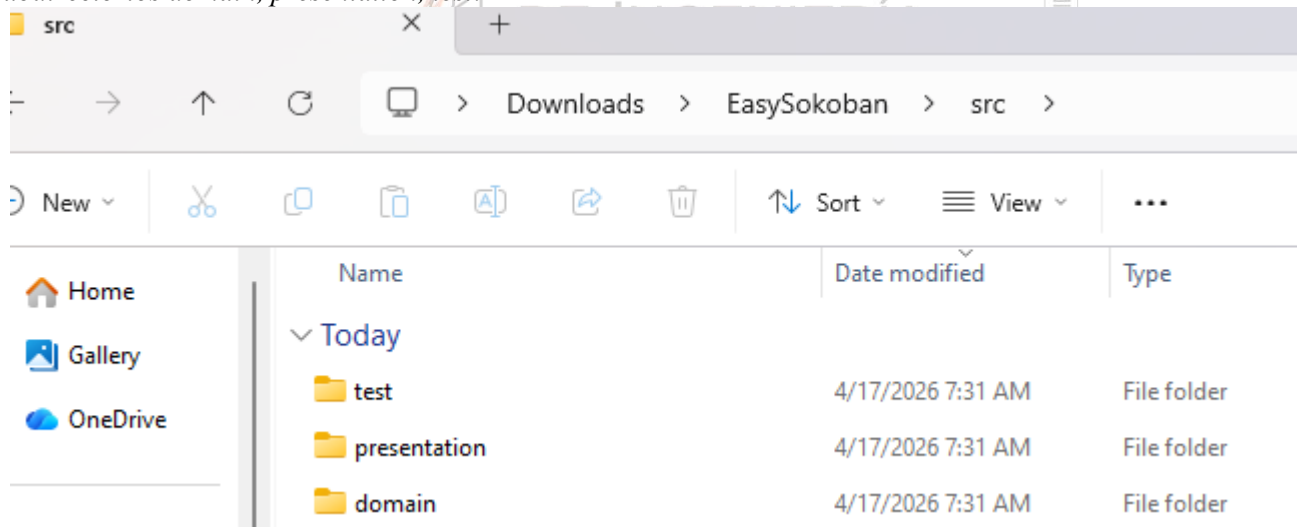
R/: Directorio EasySokoban:

EasySokoban 4/17/2026 7:30 AM File folder

Directorios *src*, *bin* y *lib*:



Subdirectorios *domain*, *presentation*, *test*:

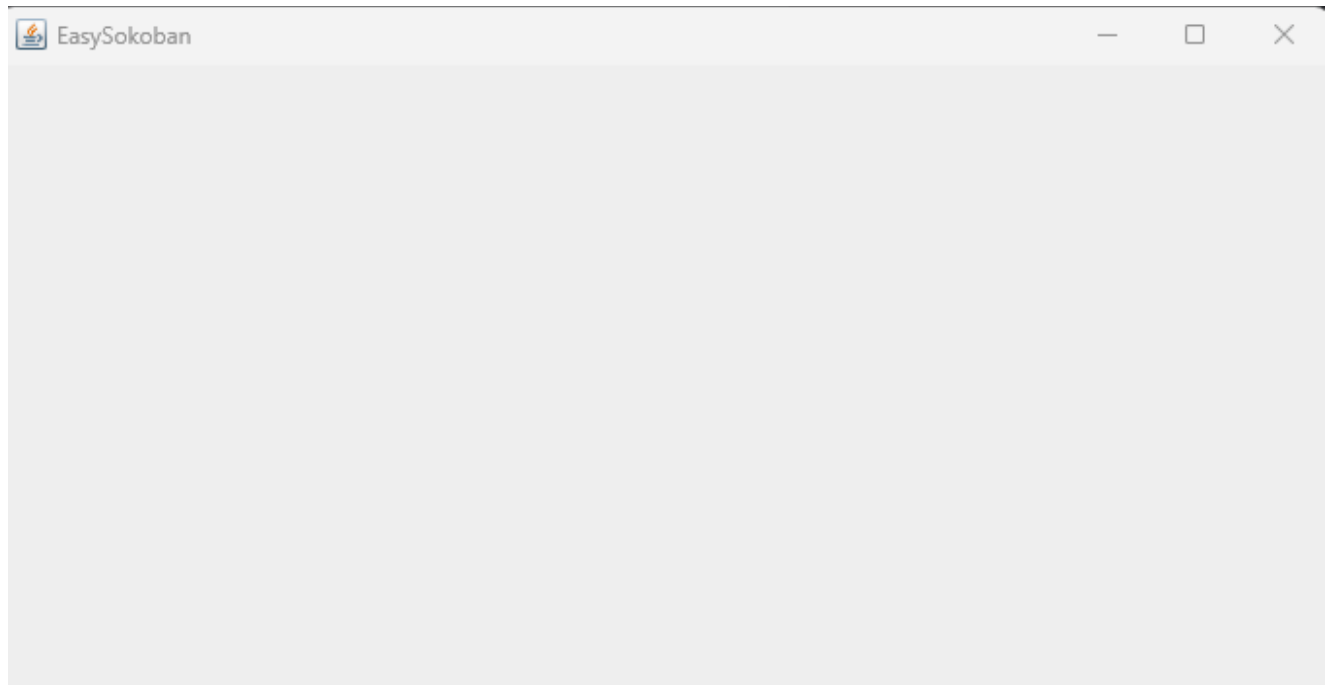


B Ciclo 0: Ventana vacía – Salir [En [lab05.doc *.java](#)]

El objetivo es implementar la ventana principal de **EasySokoban** con un final adecuado desde el icono de cerrar. Utilizar el esquema de [prepareElements-prepareActions](#).

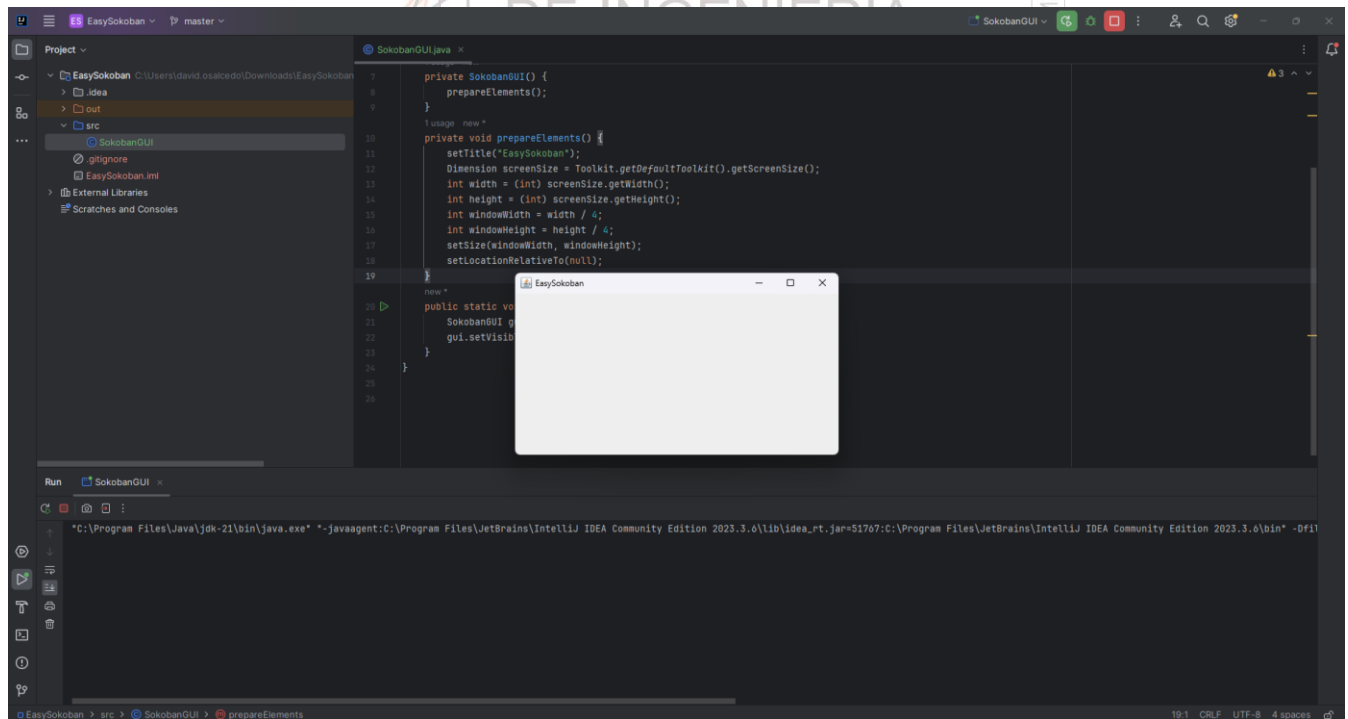
1. Construyan el primer esquema de la ventana de **EasySokoban** únicamente con el título “**EasySokoban** ”. Para esto creen la clase [SokobanGUI](#) como un [JFrame](#) con su creador (que sólo coloca el título) y el método [main](#) que crea un objeto [SokobanGUI](#) y lo hace visible. Ejecútenlo. Capturen la pantalla. (Si la ventana principal no es la inicial en su diseño, después deberán mover el [main](#) al componente visual correspondiente)

R/:



2. Modifiquen el tamaño de la ventana para que ocupe un cuarto de la pantalla y ubíquela en el centro. Para eso inicien la codificación del método `prepareElements`. Capturen esa pantalla.

R/:



3. Traten de cerrar la ventana. ¿Termina la ejecución?

R/: No termina la ejecución, pero sí se cierra la ventana.

¿Qué deben hacer en consola para terminar la ejecución?

R/:

4. Estudien en `JFrame` el método `setDefaultCloseOperation`.

¿Para qué sirve?

R/: Sirve para definir el comportamiento de una ventana cuando el usuario desea cerrarla.

¿Cómo lo usarían si queremos confirmar el cierre de la aplicación?

R/: Cuando el usuario de click al botón de cerrar, aparece un frame que le pregunta si realmente desea cerrar la ventana.

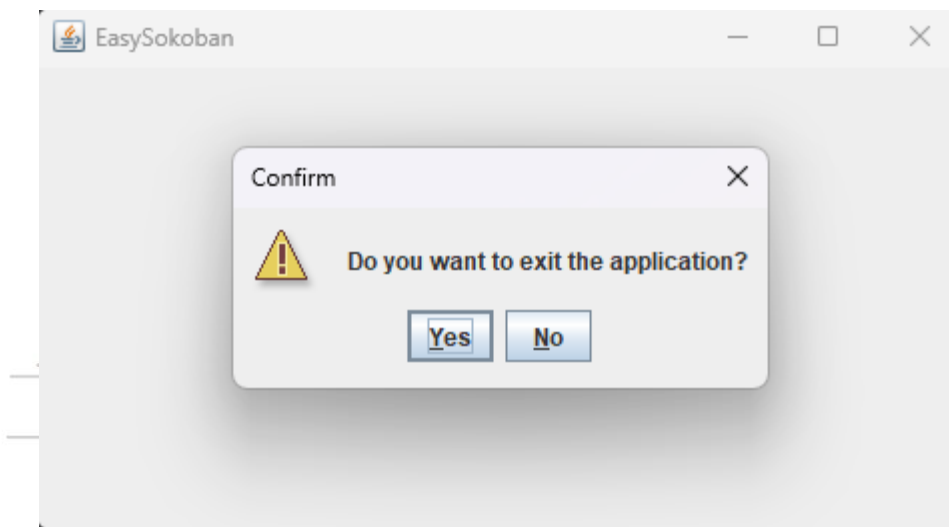
¿Cómo lo usarían si queremos simplemente cerrar la aplicación?

R/: Que la ventana se cierre cuando el usuario de click en cerrar.

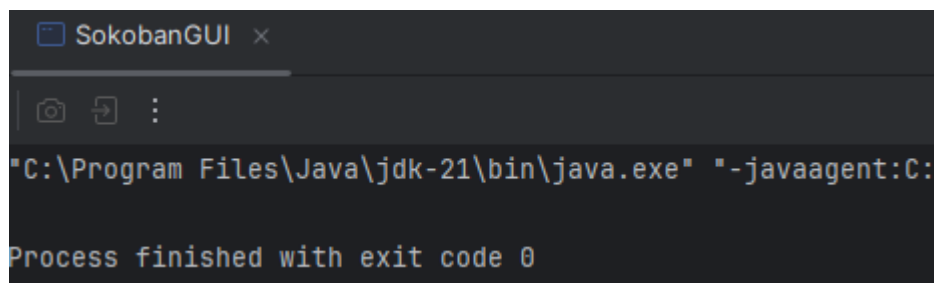
5. Preparen el “oyente” correspondiente al icono cerrar que le pida al usuario que confirme su selección. Para eso inicien la codificación del método `prepareActions` y el método asociado a la acción (`exit`). Ejecuten el programa y cierren el programa. Capturen las pantallas.

R/:

Confirmación de selección:



Cierre del programa:



C Ciclo 1: Ventana con menú – Salir [En `lab05.doc *.java`]

El objetivo es implementar un menú clásico para la aplicación con un final adecuado desde la opción del menú para salir. El menú debe ofrecer mínimo las siguientes opciones **Nuevo, Abrir – Salvar y Salir**. Incluyan los separadores de opciones.

1. Expliquen los componentes visuales necesarios para este menú. ¿Cuáles serían atributos y cuáles podrían ser variables del método `prepareElements`? Justifique.

R/: Atributos:

- sokobanMenu
- itemNew
- itemOpen
- itemSave
- itemExit

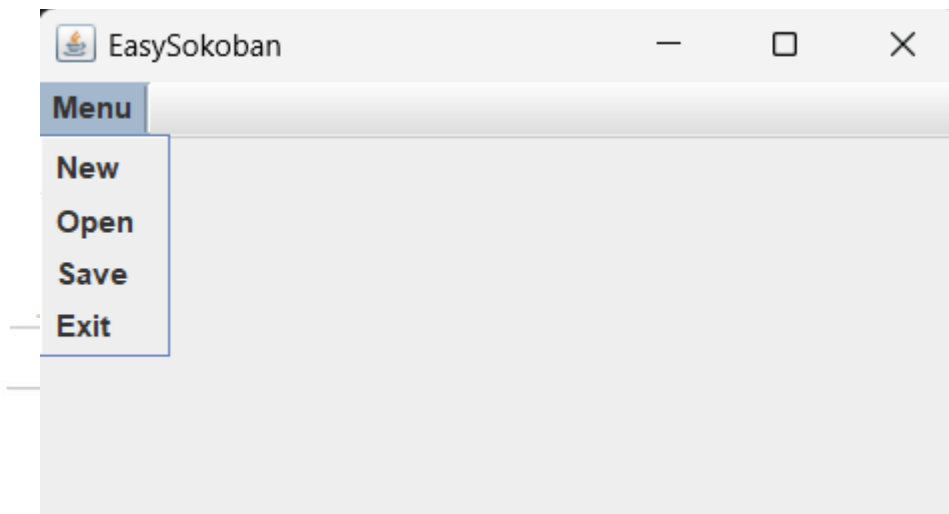
Variables:

- menu

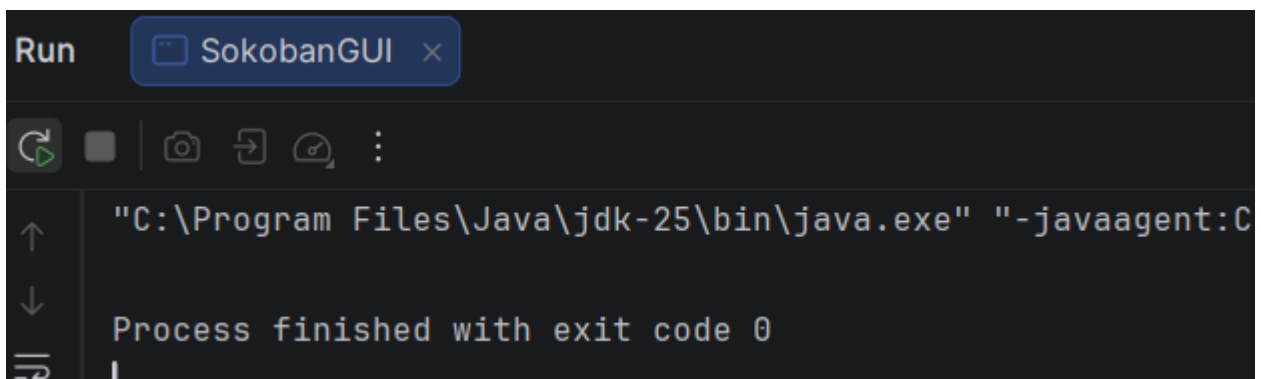
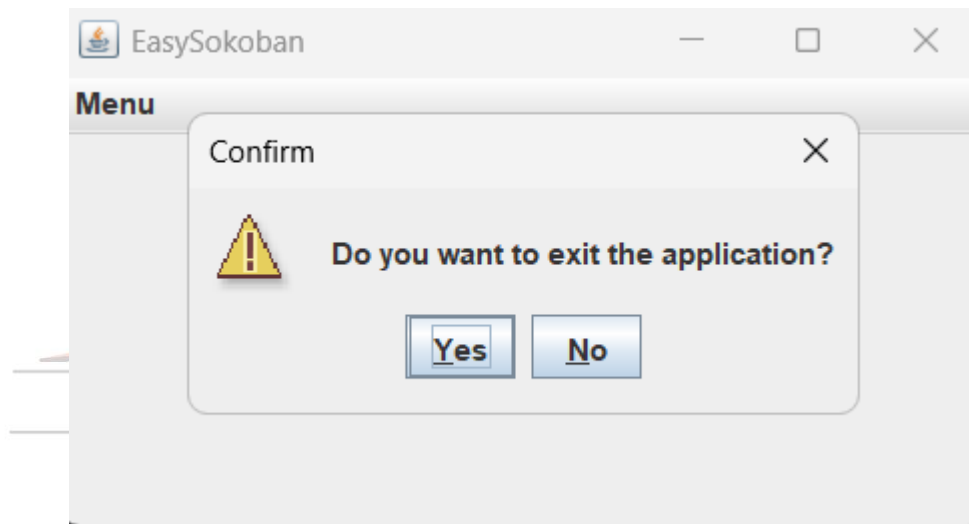
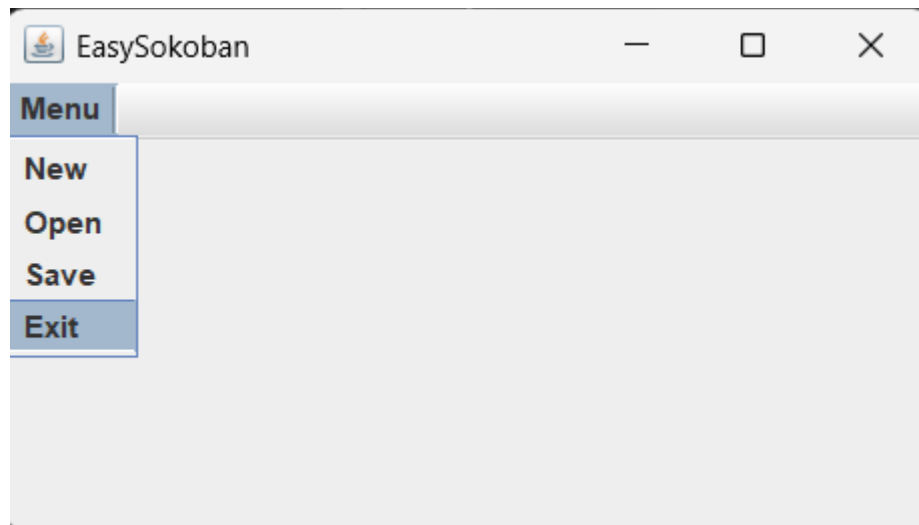
Se usaron estos con el fin de definir cada componente respecto a cada funcionalidad para una mejor claridad en el código.

2. Construya la forma del menú propuesto (`prepareElements` - `prepareElementsMenu`) . Ejecuten. Capturen la pantalla.

R/:



3. Preparen el “oyente” correspondiente al icono cerrar con confirmación (`prepareActions` - `prepareActionsMenu`). Ejecuten el programa y salgan del programa. Capturen las pantallas.



D Ciclo 2: Salvar y abrir [En lab05.doc *.java]

El objetivo es preparar la interfaz para las funciones de persistencia.

1. Detalle el componente `JFileChooser` especialmente los métodos: `JFileChooser`, `showOpenDialog`, `showSaveDialog`, `getSelectedFile`.

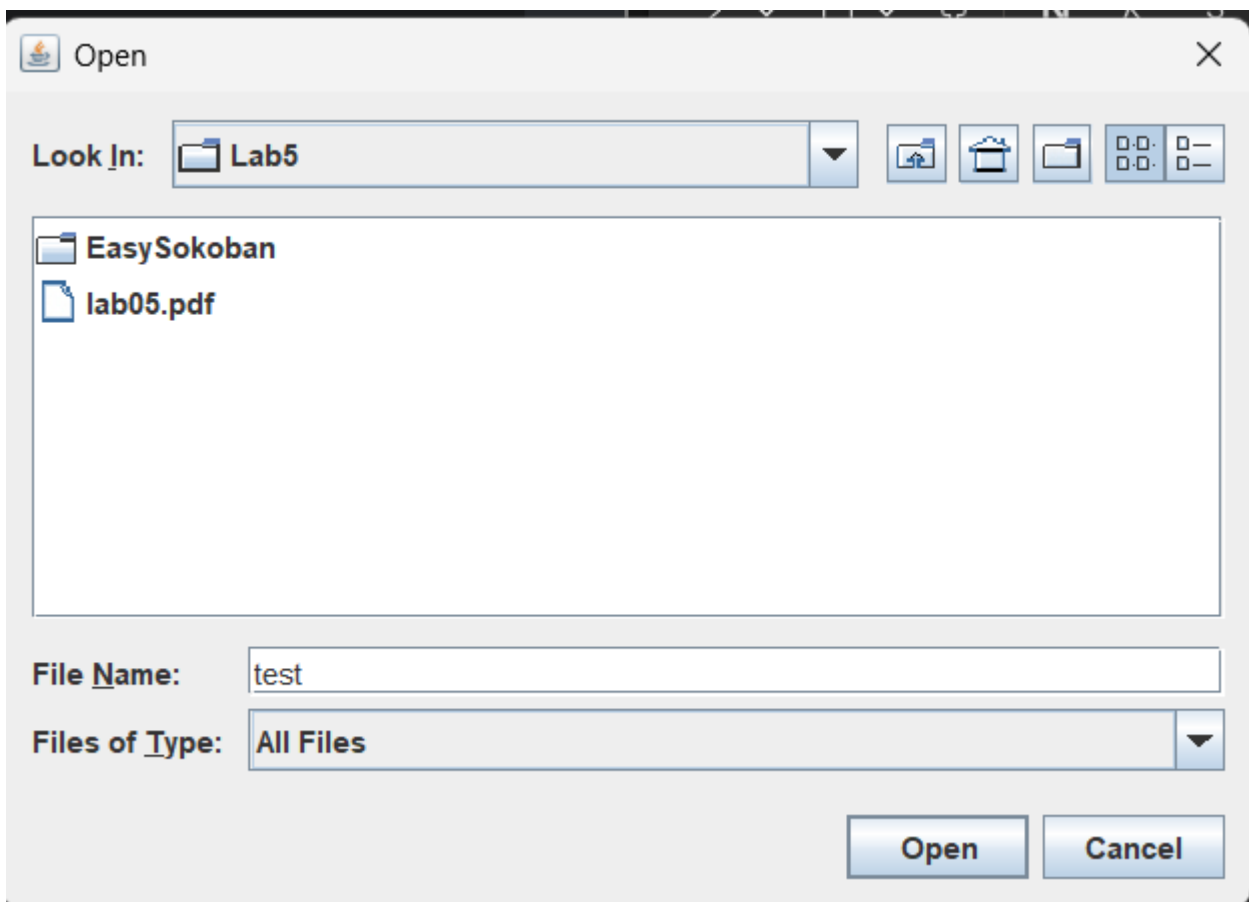
R/:

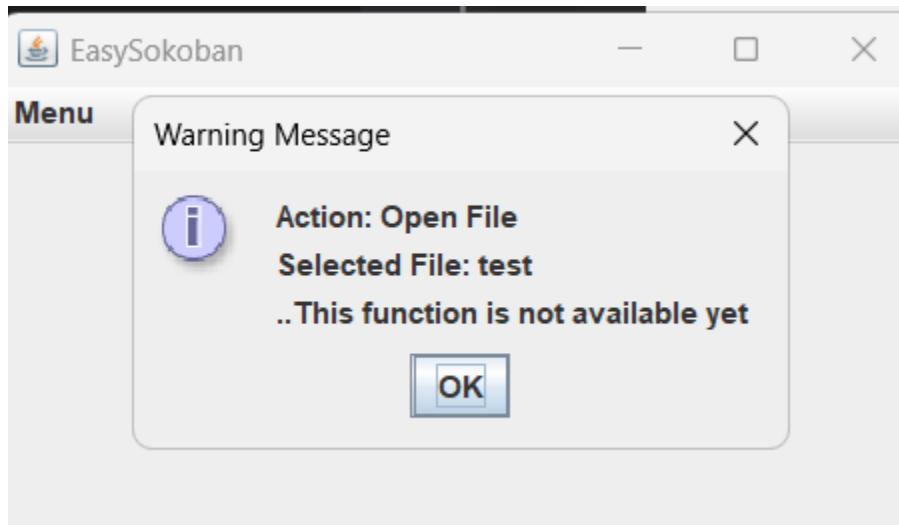
El componente JFileChooser permite al usuario abrir, crear, borrar archivos. Sus métodos son:

- **JFileChooser:** Es el constructor que permite navegar a través de los archivos locales o directorios del usuario.
 - **showOpenDialog:** Abre una ventana para seleccionar un archivo existente.
 - **showSaveDialog:** Abre una ventana que permite guardar un archivo.
 - **getSelectedFile:** Obtiene el archivo que el usuario eligió.
2. Implementen parcialmente los elementos necesarios para salvar y abrir. Al seleccionar los archivos indique que las funcionalidades están en construcción detallando la acción y el nombre del archivo seleccionado.
 3. Ejecuten las dos opciones y capturen las pantallas más significativas.

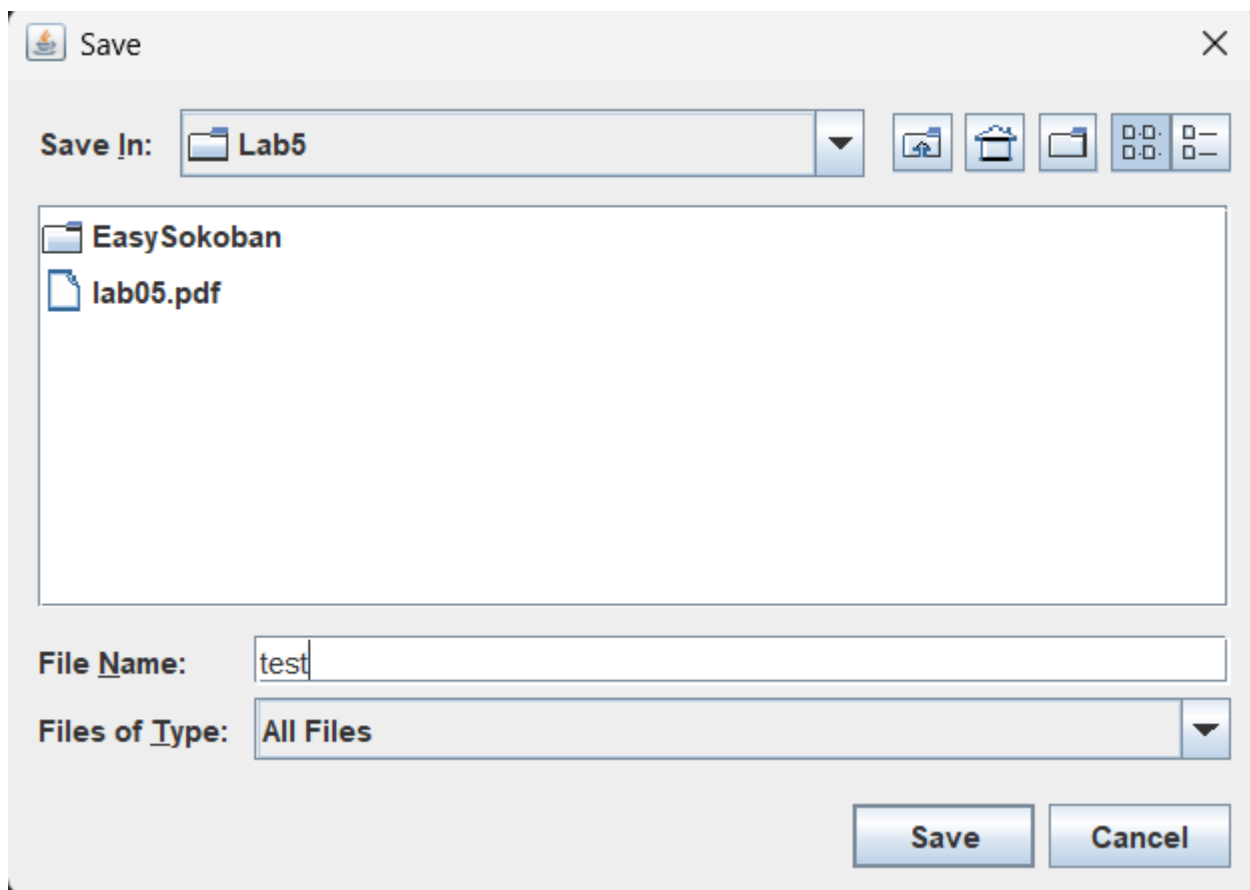
R/:

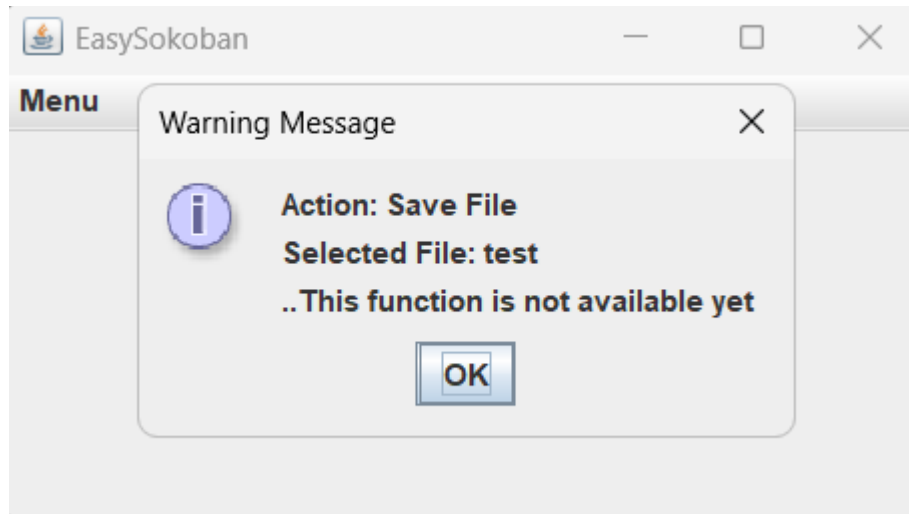
Usando el botón open:





Usando el botón save:





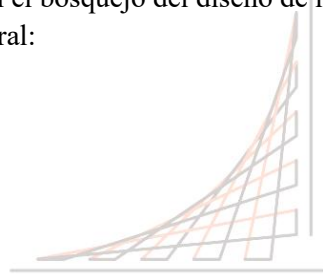
E Ciclo 3: Forma de la ventana principal

[En **lab05.doc *.java**]

El objetivo es codificar el diseño de la ventana principal (todos los elementos de primer nivel)

1. Presenten el bosquejo del diseño de interfaz con todos los componentes necesarios. (Incluyan la imagen)

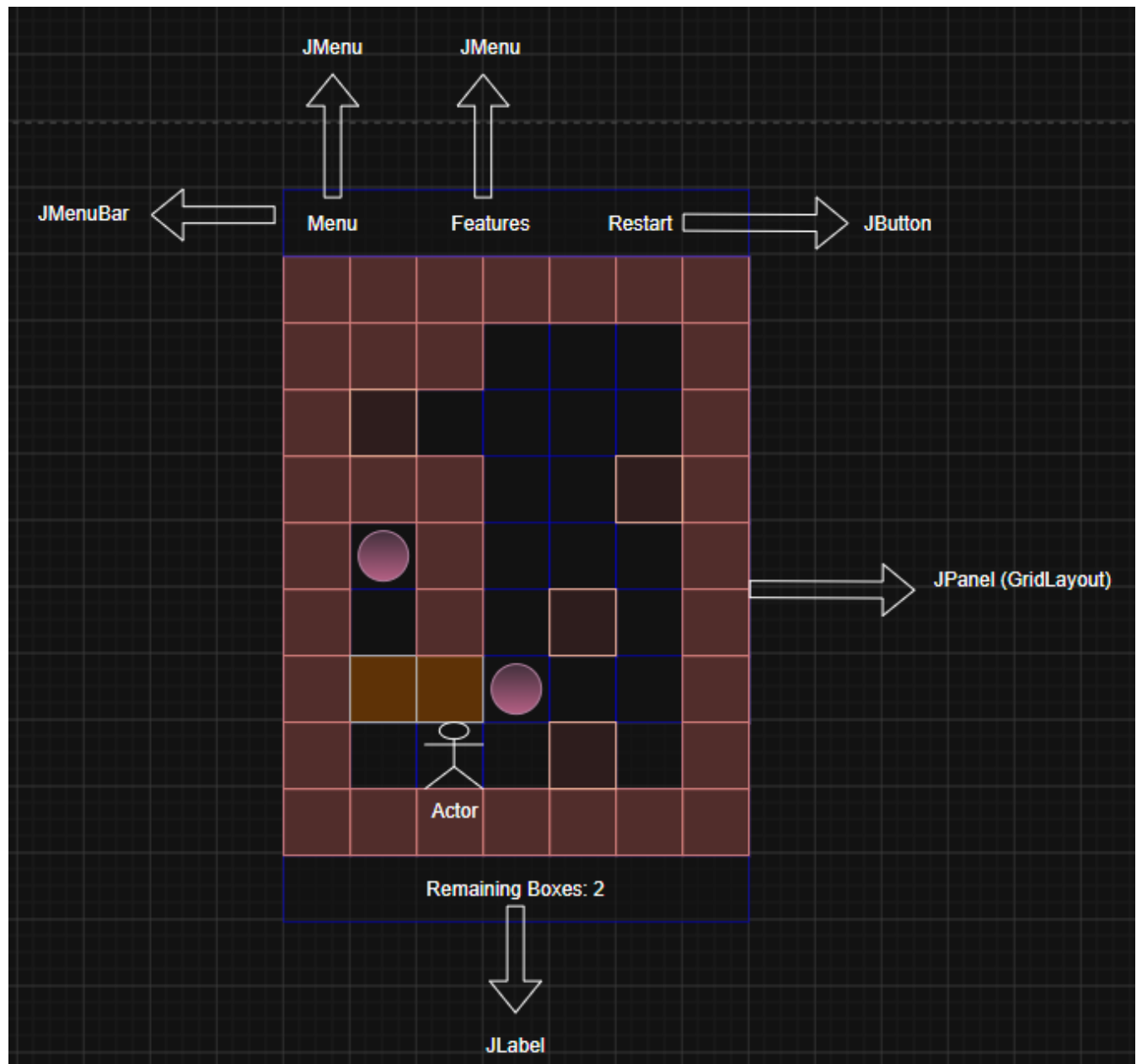
Diseño general:



ESCUELA
COLOMBIANA
DE INGENIERÍA
JULIO GARAVITO

VIGILADA MINEDUCACIÓN

UNIVERSIDAD



Diseño mVc:

- **JMenuBar**
- **JMenu:** Menu, Features
- **JMenuItem:** New, Open, Save, Exit, Color (boxes and destinations), Size (board)
- **JButton:** Restart
- **JPanel:** Tablero del juego en estilo GridLayout.
- **JLabel:** Etiqueta que muestra las cajas restantes a llegar a los destinos.

Diseño mvC:

Movimiento jugador:

- **Evento:** KeyPressed
- **Oyente:** KeyListener

- **Acción:** El jugador puede moverse si la casilla está vacía o mover la caja si no hay una pared que obstaculice a la caja

Guardar o abrir un juego:

- **Evento:** ActionPerformed
- **Oyente:** ActionListener
- **Acción:** Llamar a un JFileChooser para abrir o cargar el estado de un objeto Sokoban en un archivo
- **Cambiar colores:**
- **Evento:** ActionPerformed
- **Oyente:** ActionListener
- **Acción:** Cambiar el color a las cajas que vienen por defecto como cafés y naranjas, y los destinos como rosas, donde se refresca la interfaz (método casi implementado en el siguiente punto)

Cambiar tamaño del tablero

- **Evento:** ActionPerformed
- **Oyente:** ActionListener
- **Acción:** Cambiar el tamaño del tablero según lo quiera el jugador, donde se tiene en cuenta que no pueden ser valores negativos, ni un tablero muy pequeño para poder jugar.

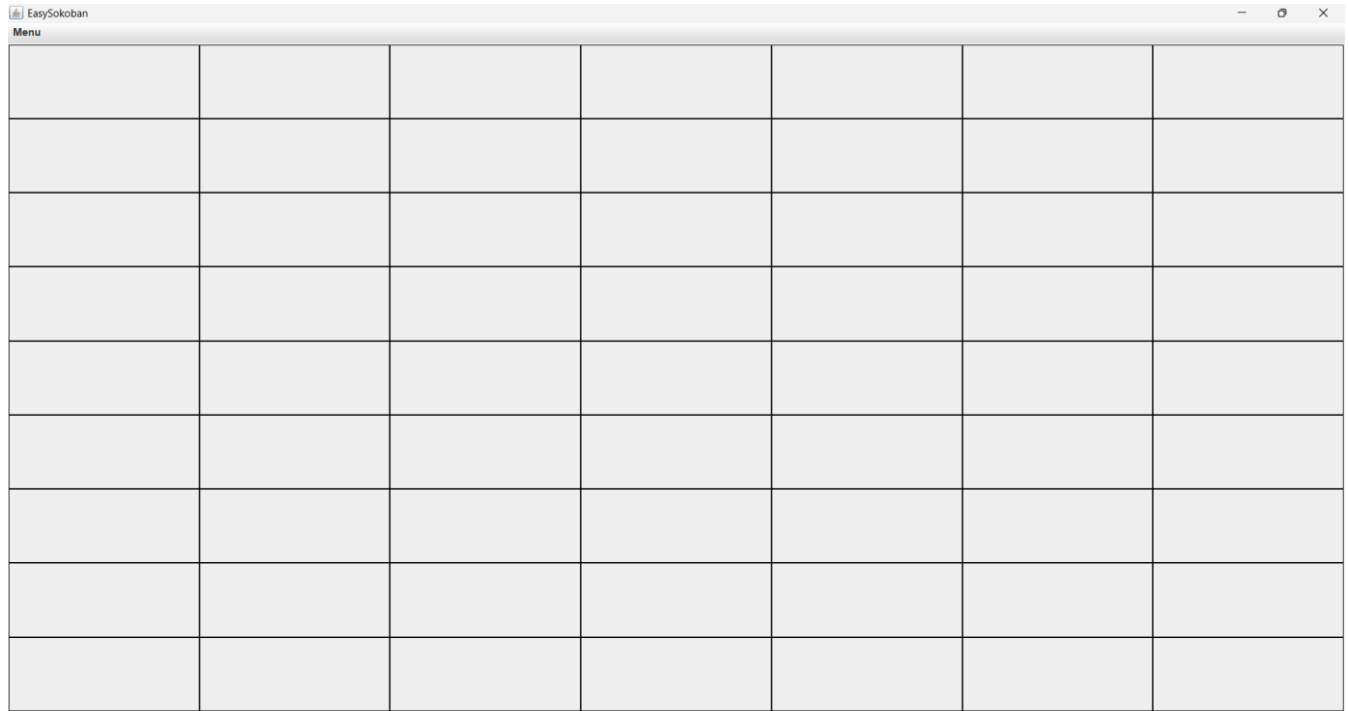
Diseño Mvc:

Sokoban

- board
- height, width
- playerX, playerY
- boxColor, destinationColor
- movePlayer(int height, int width)
- countBoxesOnDestination()
- IsLevelCompleted()

2. Continúen con la implementación definiendo los atributos necesarios y extendiendo el método [prepareElements](#). Para la zona del tablero definan un método [prepareElementsBoard](#) y un método [refresh](#) que actualiza la vista del tablero considerando, por ahora, el tablero inicial por omisión. Este método lo vamos a implementar realmente en otros ciclos.
3. Ejecuten y capturen la pantalla.

R/:



F Ciclo 4: Cambiar colores

[En [lab05.doc *.java](#)]

El objetivo es implementar este caso de uso.

1. Expliquen los elementos (vista – controlador) necesarios para implementar este caso de uso.

R/:

Vista (View):

JMenuItem (itemchangeBoxColor e changeDestinationColor): Son los elementos en el menú Archivo que el usuario visualiza y utiliza para iniciar la acción.

JColorChooser: Un componente estándar de Java Swing que actúa como una vista emergente (diálogo) ofreciendo una interfaz rica para seleccionar colores (por muestras, HSB, RGB, etc.).

Tablero (boardPane y método refresh() en SokobanGUI): La vista principal que se encarga de redibujar cada panel (JPanel) utilizando los colores guardados en las variables boxColor y playerColor.

Controlador (Controller):

ActionListener: Son los oyentes (definidos mediante expresiones lambda en prepareActionsMenu()) que interceptan el evento de clic en los elementos del menú. Estos actúan como intermediarios: abren el JColorChooser, capturan el color seleccionado, actualizan el estado interno de la interfaz (las variables de color) y finalmente ordenan a la vista repintarse llamando a refresh().

2. Detalle el comportamiento de JColorChooser especialmente el método estático showDialog.

R/: El método estático JColorChooser.showDialog(Component component, String title, Color initialColor) tiene el siguiente comportamiento específico:

Bloqueo Modal: Crea y muestra un cuadro de diálogo modal. Esto significa que mientras el diálogo de selección de color esté abierto, el usuario no podrá interactuar con la ventana principal de la aplicación (SokobanGUI).

Parámetros:

component: Define el componente padre. El diálogo aparecerá centrado en relación con este componente (en nuestro caso, this, la ventana principal).

title: Es el texto que se muestra en la barra de título de la ventana del diálogo (ej: "Seleccionar Color Cajas").

initialColor: Es el color que aparecerá preseleccionado cuando se abra la ventana. Esto ayuda al usuario a saber cuál es el color actual antes de cambiarlo.

Retorno:

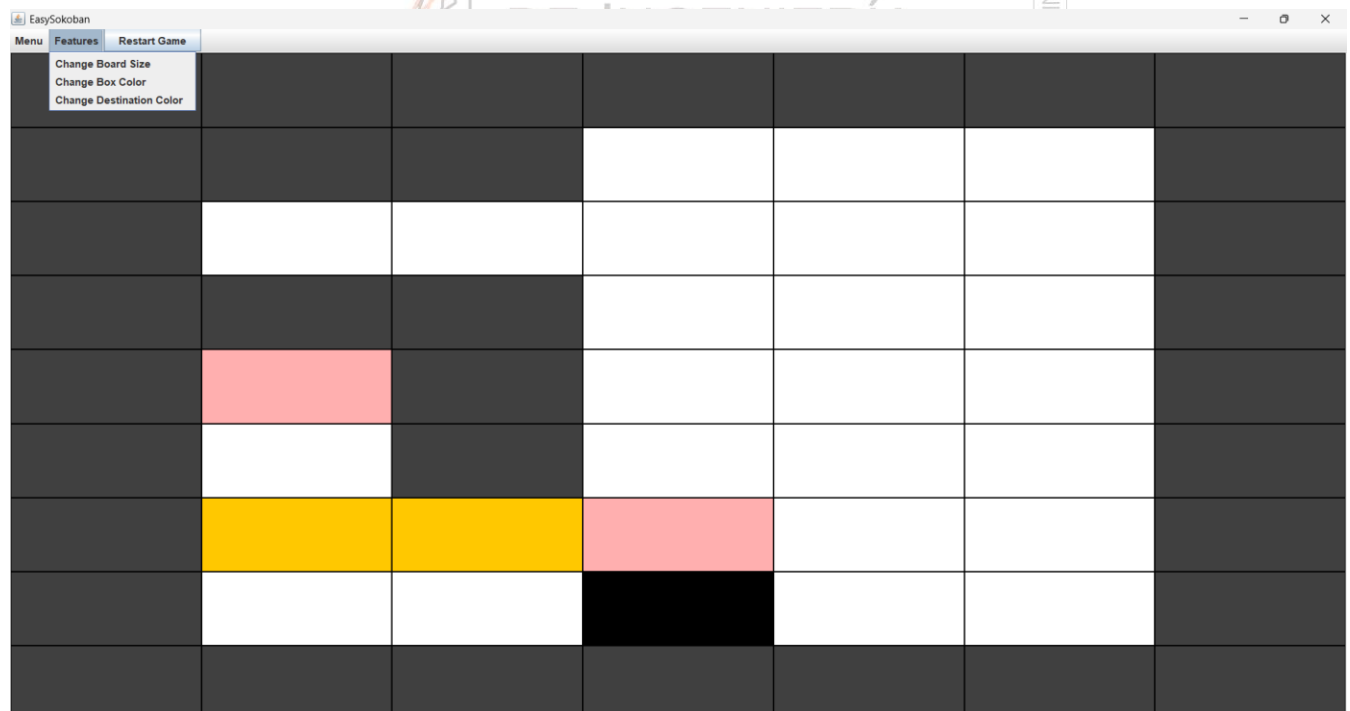
Si el usuario elige un color y hace clic en "Aceptar" (OK), el método devuelve un objeto de tipo Color que representa la elección.

Si el usuario hace clic en "Cancelar" o cierra la ventana (con la 'X'), el método devuelve null. Por esto es importante la validación `if (color != null)` antes de aplicar los cambios.

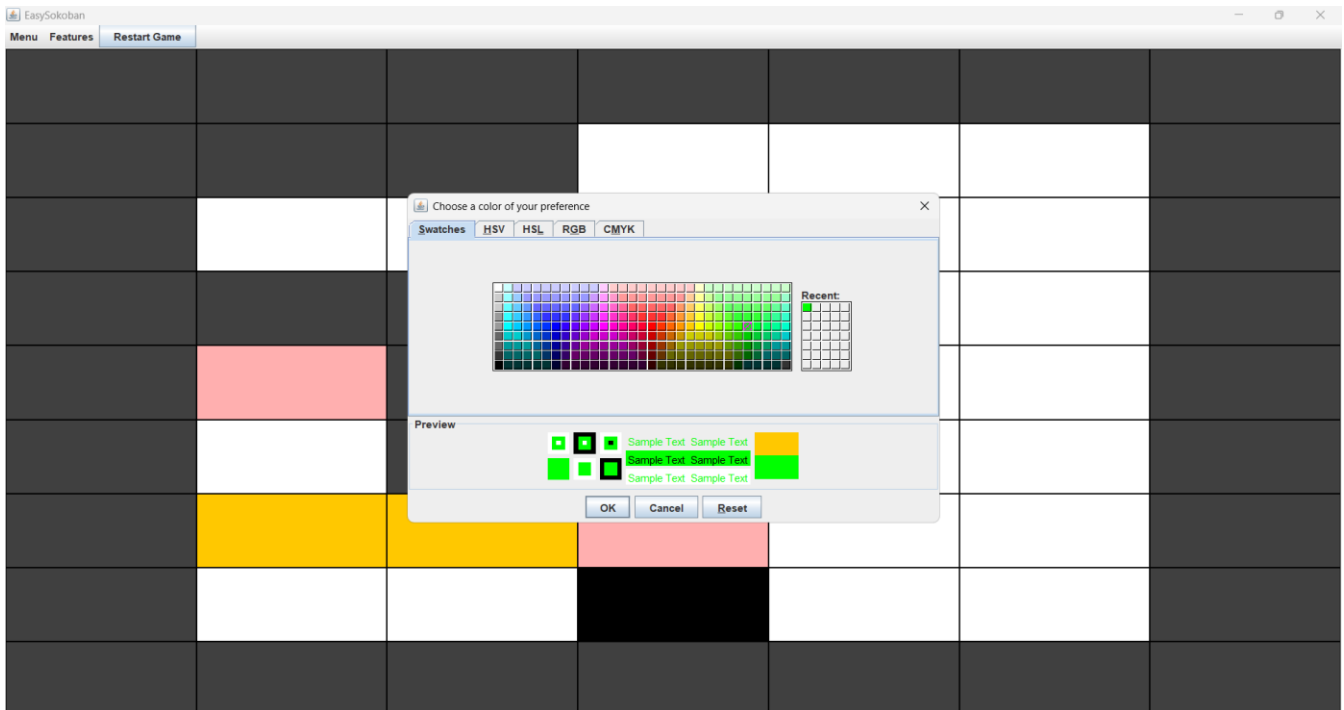
3. Implementen los componentes necesarios para cambiar el color de las piezas.

4. Ejecuten el caso de uso y capture las pantallas más significativas.

Cajas naranjas y destinos rosas (de acuerdo con el papel dado en el laboratorio):



Al escoger un color para una caja/destino:



Cajas verdes y destinos rojos:



PARTE II. Funcionalidades básicas

A Ciclo 5: Modelo Sokoban [En lab05.doc *.java]

(NO OLVIDE BDD – MDD)

El objetivo es implementar la capa de dominio para **EasySokoban** .

1. Construya los métodos básicos del juego.
2. Ejecuten las pruebas y capturen el resultado.

✓ SokobanTest (test)	44 ms
✓ shouldNotHaveBoxesAtALevelStart()	30 ms
✓ shouldValidateBoardSize()	4 ms
✓ shouldNotMoveThroughWalls()	3 ms
✓ shouldNotCreateABoardWithNegativeValues()	7 ms

B Ciclo 6: Jugar [En lab05.doc *.java]

El objetivo es implementar el juego

1. Expliquen los elementos necesarios para implementar este caso de uso.

R/:

- **JPanel** cells
- **JMenuBar** sokobanMenu
- **JMenu** menu, features
- **JButton** restartButton
- **Sokoban** sokoban

Estos fueron los elementos necesarios para implementar la jugabilidad del juego, dado que se necesita una ventana principal para que el juego se abra, el panel para crear el tablero, la barra de menú para distribuir los distintos menús que ofrece el juego: Menú con configuraciones y menú con características que el jugador quiera cambiar; así mismo, el botón para reiniciar el juego donde prácticamente retorna al jugador a la posición original del nivel por defecto (crea un nuevo Sokoban).

2. Implementen los componentes necesarios para jugar .¿Cuántos oyentes necesitan? ¿Por qué?

R/:

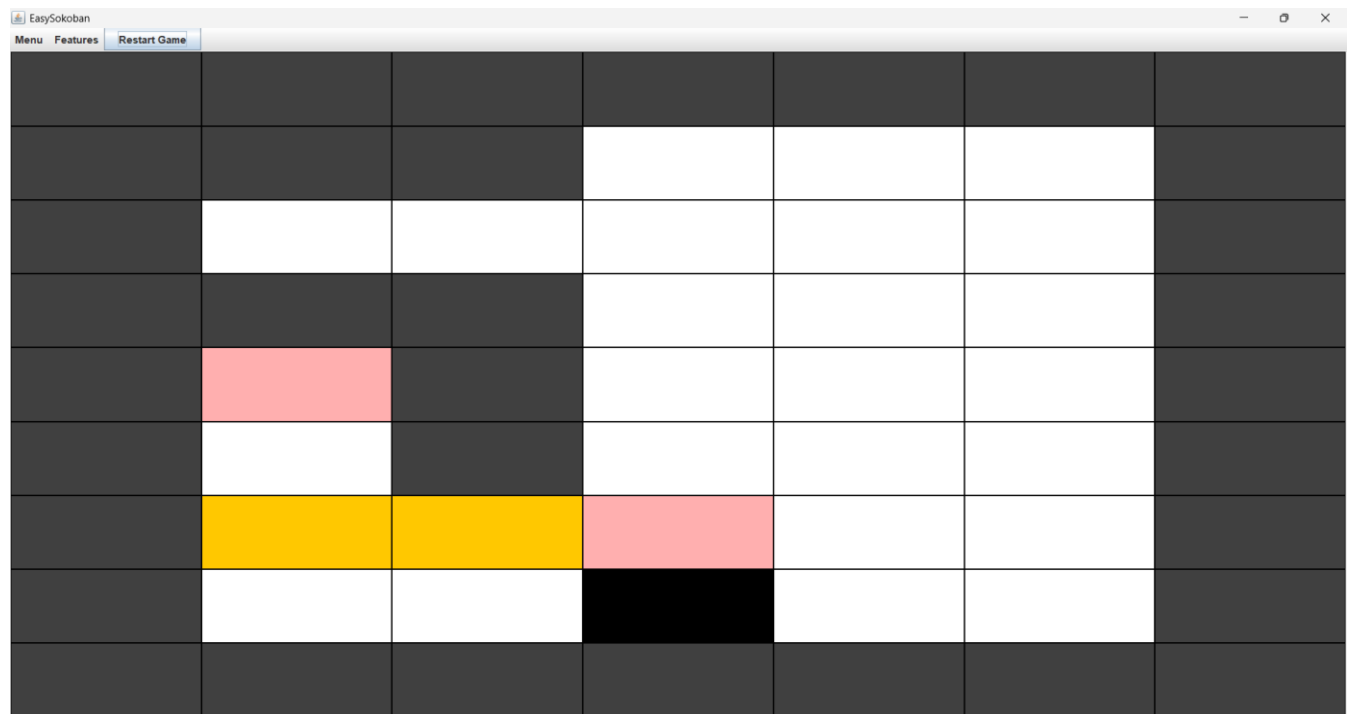
- **WindowListener**
- **ActionListener**

Se necesitan 2 oyentes, principalmente para mostrar la ventana de archivos locales que tiene el usuario. Seguidamente, las acciones que llaman a los métodos que hacen la lógica para mover el jugador, cambiar el color de una caja o destino, etc.

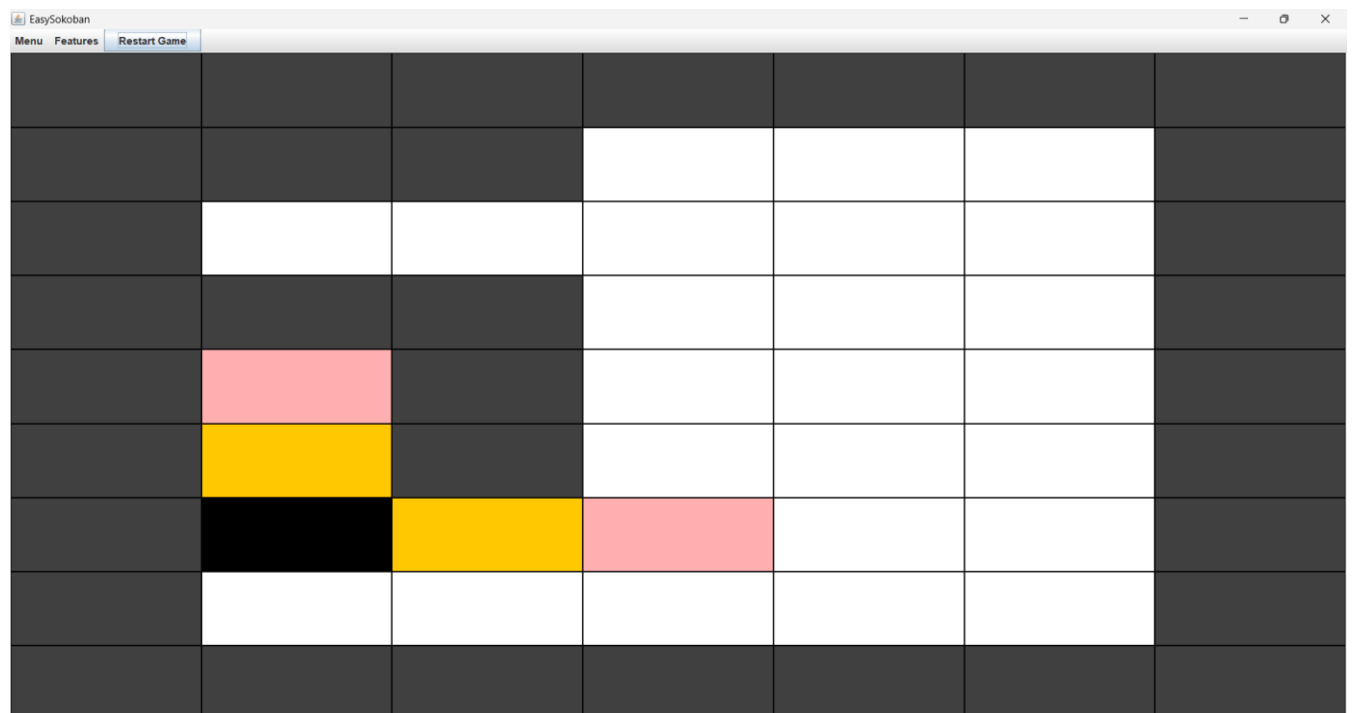
3. Ejecuten el caso de uso y capture las pantallas más significativas.

En este caso de uso, se implementó el caso de ejemplo dado a papel en el laboratorio:

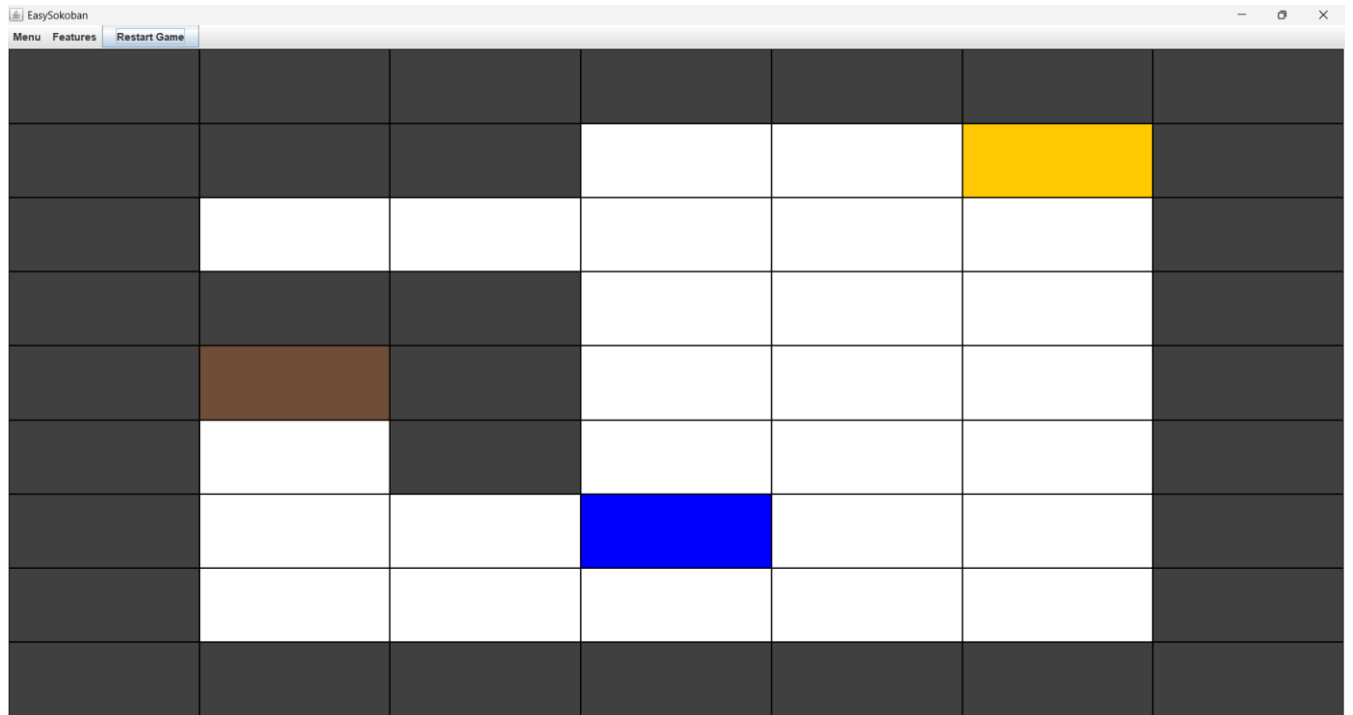
Estado inicial:



El jugador (rectángulo negro) mueve una caja:



Jugador en la posición del destino (rectángulo azul):



Cajas en sus destinos (se vuelven cafés):



PARTE III. BONO: Funcionalidades complementarias

A Ciclo 7: Reiniciar [En lab05.doc *.java]

El objetivo es implementar este caso de uso.

1. Expliquen los elementos a usar para implementar este caso de uso.

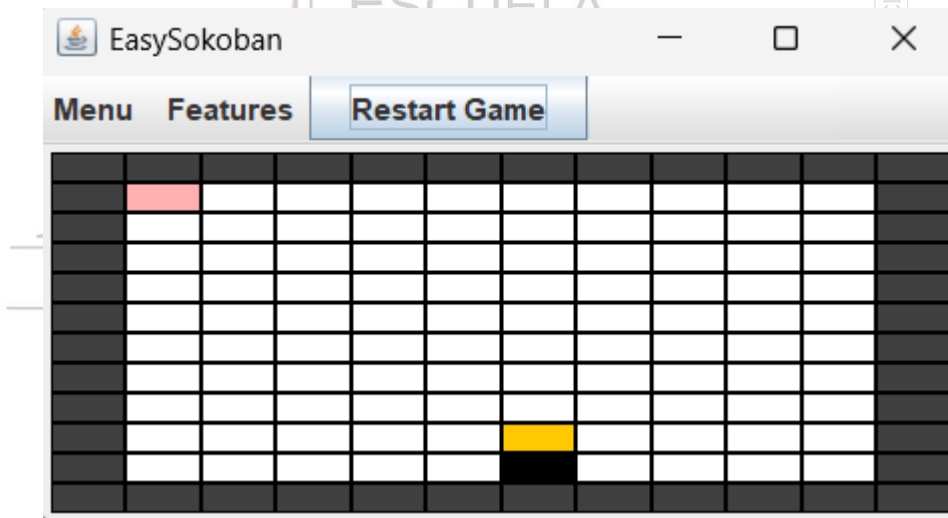
R/: Los elementos para implementar este caso de uso son:

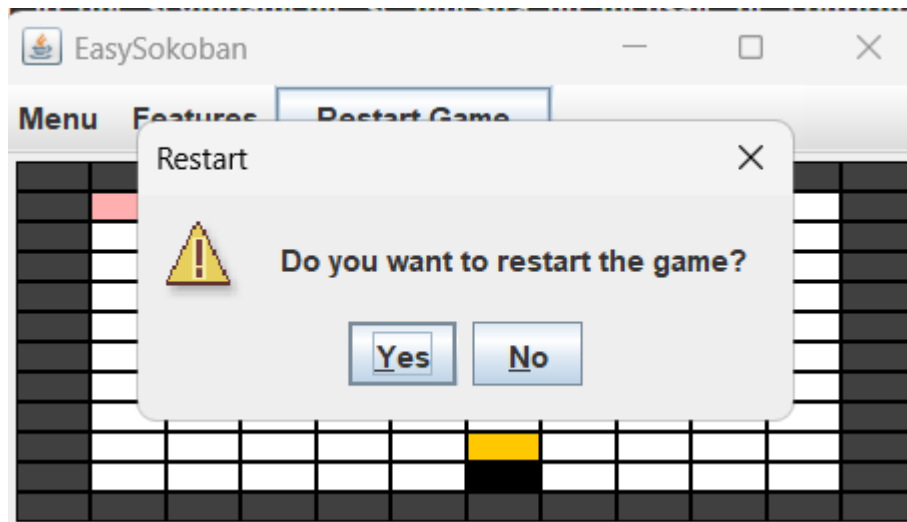
- **JButton** restartButton
- **JOptionPane**

Se usarán estos componentes con el fin de que haya un oyente cuando el jugador haga click en el botón de 'Restart Game', por lo que seguidamente se muestra un mensaje de confirmación que le pregunta si realmente quiere reiniciar el juego. De manera que cuando se reinicie el juego, la interfaz deja el tablero y demás objetos a una posición por defecto; así mismo, decidimos dejar esa función aparte del menú Features para que sea una visualización directa de interfaz a jugador.

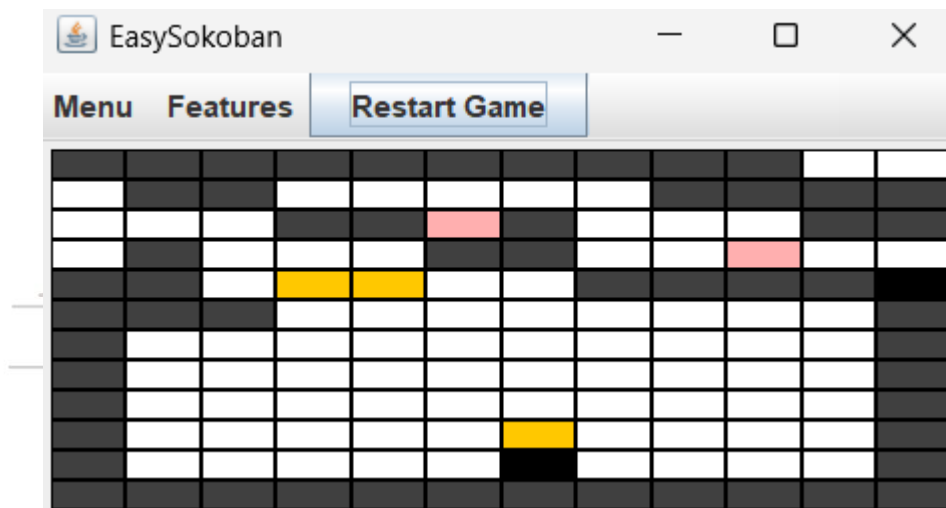
2. Implementen los elementos necesarios para reiniciar
3. Ejecuten el caso de uso y capture las pantallas más significativas.

Primeramente, se cambia el tamaño del tablero para una mejor visualización de lo que sucederá:





Juego reiniciado:



B Ciclo 8: Cambiar el tamaño [En lab05.doc *.java]

El objetivo es implementar este caso de uso.

1. Expliquen los elementos a usar para implementar este caso de uso

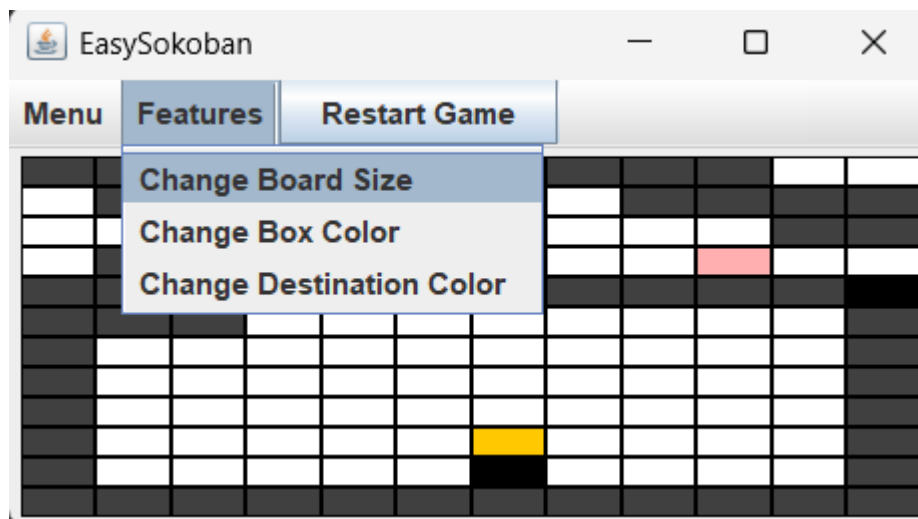
R/:

- **JMenuItem** itemChangeBoardSize
- **JOptionPane**

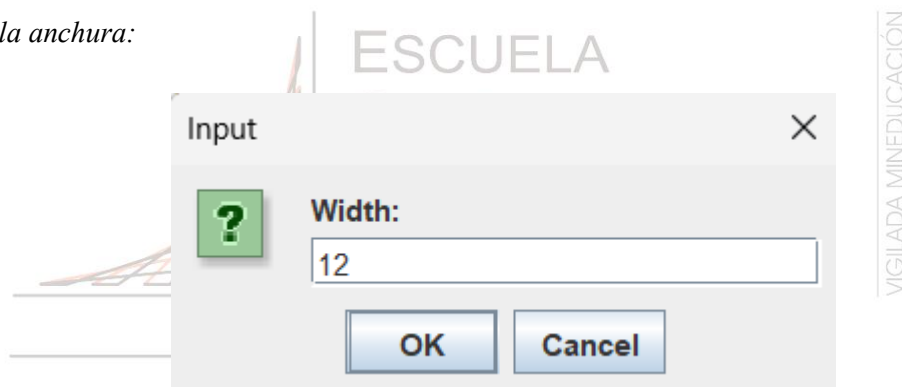
Se usarán estos dos componentes, ya que se quiere implementar un ítem adicional dentro del menú Features (recordando que este menú contiene partes funcionales que el jugador puede cambiar a su gusto) y así mismo, donde se le pregunte la anchura y altura que desee cambiar al tablero donde el nivel cambia de acuerdo con lo que el jugador ingrese. Finalmente, si llega a poner algún valor menor a 3 o mayor a 20, se le mostrará un mensaje de que el tablero es muy pequeño o grande (se siguieron las recomendaciones de la profesora María Irma).

2. Implementen los elementos necesarios para cambiar el tamaño del juego
3. Ejecuten el caso de uso y capture las pantallas más significativas.

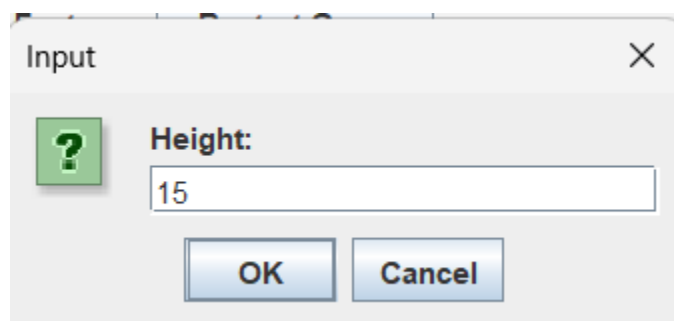
R/:



Cambiando la anchura:



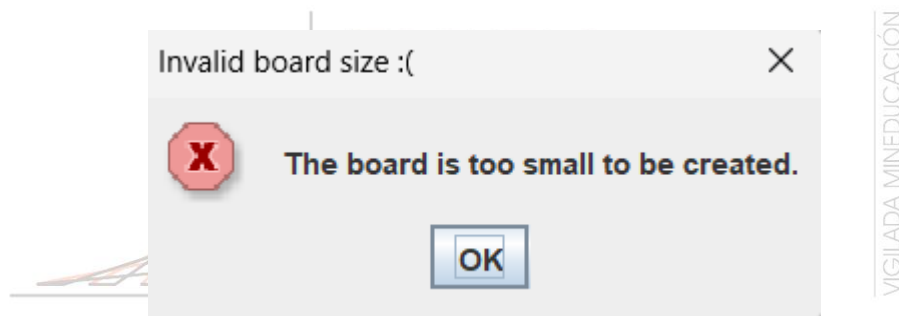
Cambiar la altura:



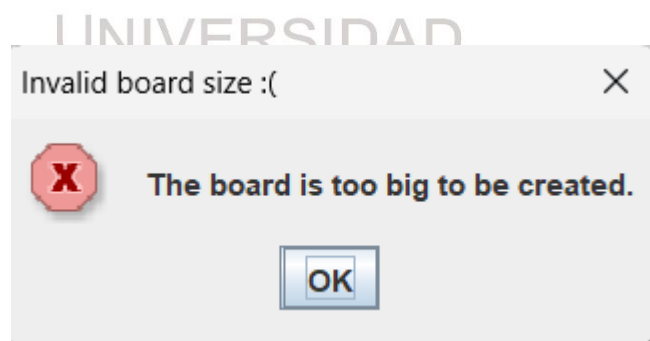
Tablero con tamaño 12x15:



Validación si es muy pequeño el tablero:



Validación si el tablero es grande:



RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas)

R/:

- **Juan Pablo Cuervo Contreras:** 32 horas.
- **David Felipe Ortiz Salcedo:** 32 horas.

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?

R/: Completo en su mayoría, ya que dedicamos el tiempo adecuado para terminar el juego de manera que quede casi funcional. Aunque faltó implementar la cantidad de cajas que el jugador puso en los destinos y que ganó al ponerlas en esas posiciones.

3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?

R/: La práctica más útil en este laboratorio fue Coding: All production code is pair programmed, Coding: Code must be written to agreed standards, dado que el laboratorio al ser demasiado extenso, el hecho de dividirnos el trabajo nos benefició mucho para poder culminarlo y seguir las instrucciones del monitor como lo fue escribir el código en un sólo idioma: inglés o español para que aquel que quiera leer el código, no tenga que exigirse demasiado.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

R/: El mayor logro fue conseguir que los objetos estén dentro de los límites y además de que el jugador se pudiera mover a través del tablero de manera que no se saliera de los límites dado el tamaño que el jugador le dé o el que viene por defecto.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

R/: Consideramos que el mayor problema técnico fue tratar de adaptarnos a manejar todo por consola, ya que en BlueJ estábamos acostumbrados a crear clases, interfaces y paquetes mucho más fácil (visualmente hablando) de manera que tuvimos que darnos varios días para poder acostumbrarnos a la nueva forma de estudiar.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

R/: Dividirnos el trabajo con el fin de que cada integrante tuviera en cuenta qué se debía realizar en el laboratorio y tratar de culminarlo. Nos comprometemos a seguir realizando código más entendible donde no se usen variables que no sean descriptivas y adaptarnos al nuevo entorno de trabajo para futuros trabajos.

7. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

R/:

- W3Schools. W3Schools — Tutoriales y referencias de programación. —

<https://www.w3schools.com/>

- Rodríguez, A. (s. f.). Documentar proyectos Java con Javadoc: Comentarios, símbolos, tags (deprecated, param, etc.) Aprender a Programar.

https://www.aprenderaprogramar.com/index.php?option=com_content&view=article&id=646:documentar-proyectos-java-con-javadoc-comentarios-simbolos-tags-deprecated-param-etc-cu00680b&catid=68&Itemid=188